

 ENGINEERING PROJECT REPORT

ChaosMesh-Lite

Network Resilience & Fault Injection Control Plane for Distributed
Microservices

Full-Stack Systems Architecture & Telemetry Analysis

DONE BY

Abhineet R

CSE Core

VIT Bhopal University

The Reality of Distributed Failures

✓ Happy Path Assumption

Predictable Latency: Millisecond roundtrip timing

Consistent Status: Reliable 200 OK responses

Persistent Sockets: Unbroken TCP streams

⚠ Production Reality

Network Jitter: High-tail p95 latency spikes

Upstream Panics: Cascading 500 error storms

Abrupt Disconnects: Dropped sockets without FIN packets

Reverse Proxy Flow & Control Plane



“1” Bidirectional Telemetry Loop: Live metric packets broadcast to Vite dashboard via WebSockets.

Three Vectors of Failure Injection



Latency Injection

Programmable delays with jitter.



HTTP Status Faults

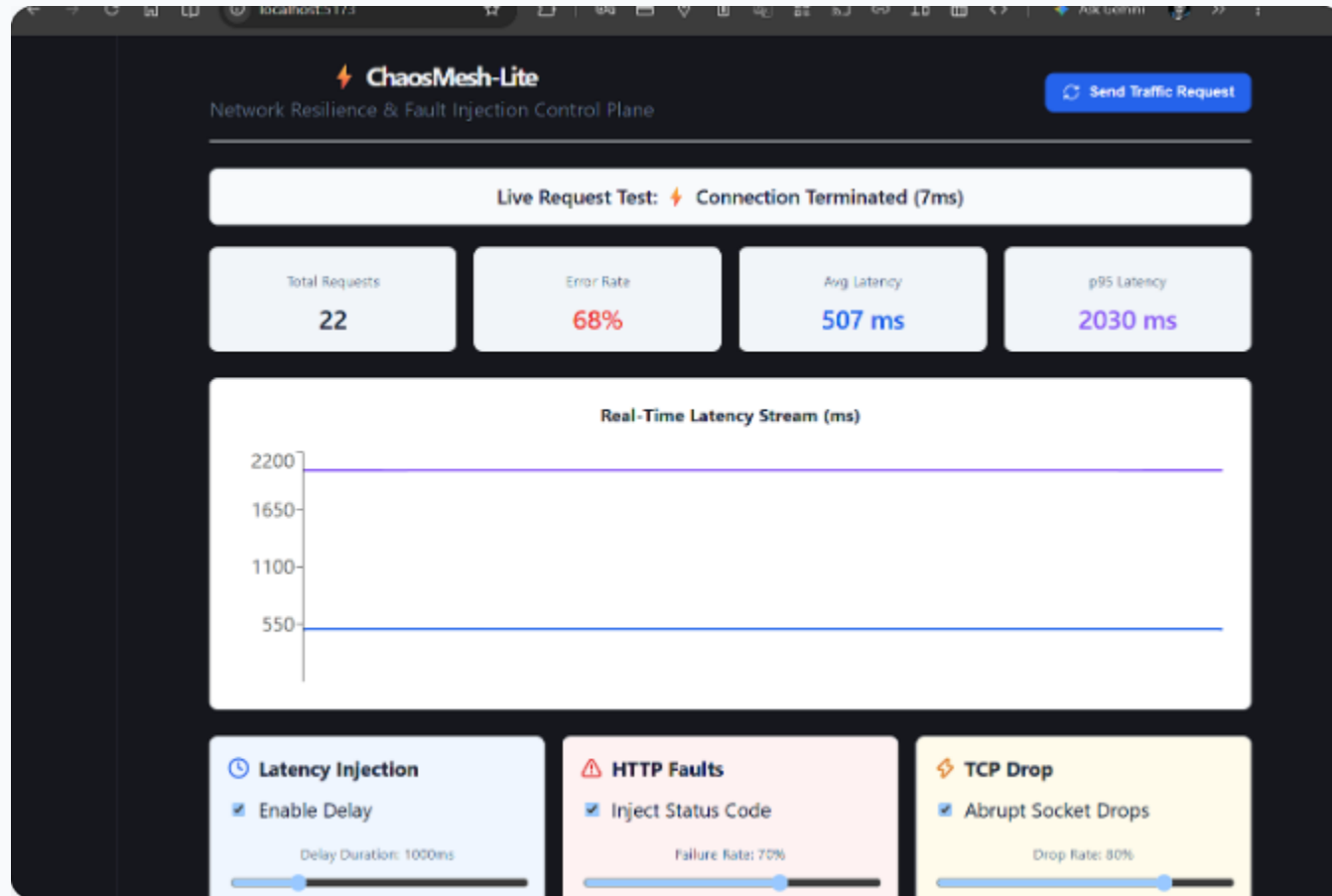
Simulated upstream 500 responses.



TCP Socket Drops

Abrupt raw socket termination.

Interactive Control Plane Dashboard



Hot Configuration

Real-time slider state updates.

Live Request Testing

Instant client test triggering.

Visual Status Banners

Immediate visual feedback loops.

Observed Benchmark Metrics

22

Total Requests Tracked

68%

Injected Error Rate

507ms

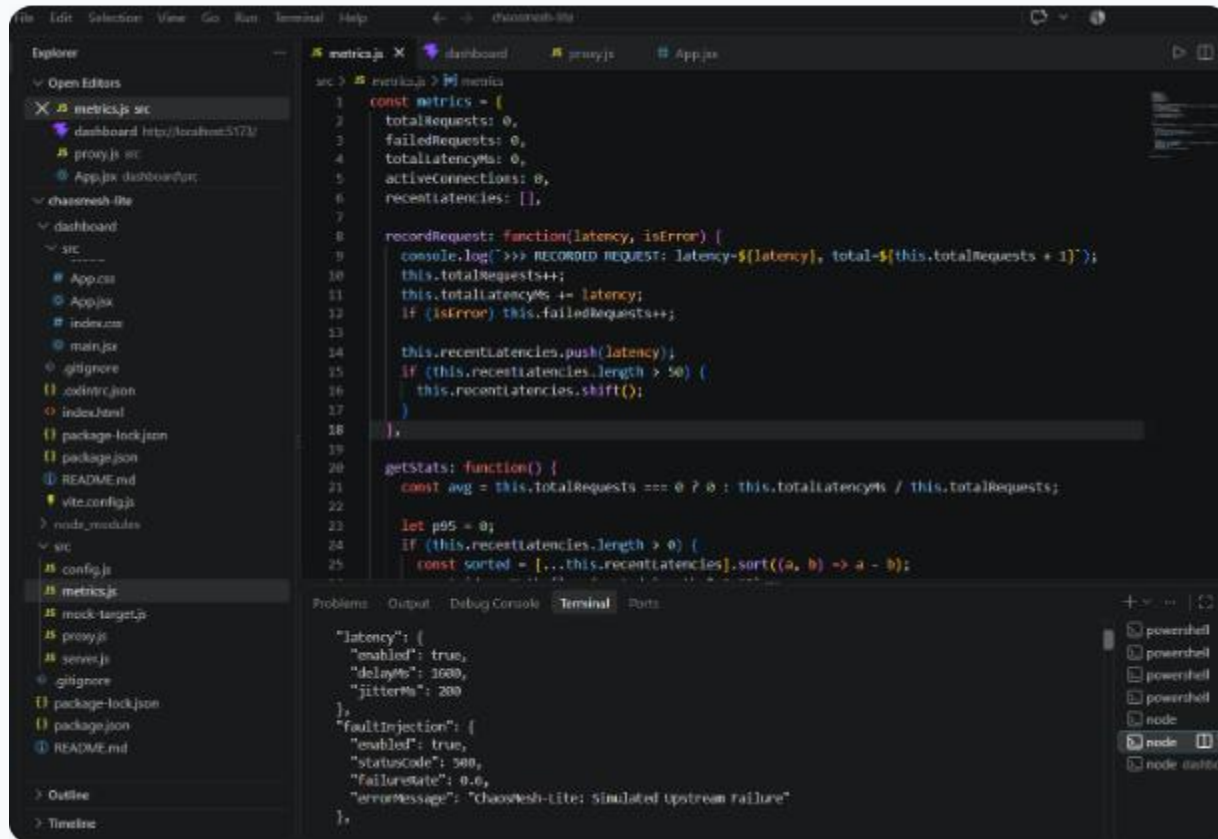
Average Response Time

2030ms

Rolling p95 Tail Latency

Live statistical aggregation captured during active fault injection routines.

In-Memory Metrics & p95 Computation



```
src > metrics.js > metrics.js
1  const metrics = {
2    totalRequests: 0,
3    failedRequests: 0,
4    totalLatencyMs: 0,
5    activeConnections: 0,
6    recentLatencies: [],
7
8    recordRequest: function(latency, isError) {
9      console.log(">>> RECORDED REQUEST: latency-${latency}, total-${this.totalRequests + 1}");
10     this.totalRequests++;
11     this.totalLatencyMs += latency;
12     if (isError) this.failedRequests++;
13
14     this.recentLatencies.push(latency);
15     if (this.recentLatencies.length > 50) {
16       this.recentLatencies.shift();
17     }
18   },
19
20   getStats: function() {
21     const avg = this.totalRequests === 0 ? 0 : this.totalLatencyMs / this.totalRequests;
22
23     let p95 = 0;
24     if (this.recentLatencies.length > 0) {
25       const sorted = [...this.recentLatencies].sort((a, b) => a - b);
26     }
27   }
28 }
```

```
Problems Output Debug Console Terminal Ports
{"latency": {
  "enabled": true,
  "delayMs": 3000,
  "jitterMs": 200
},
"faultInjection": {
  "enabled": true,
  "statusCode": 500,
  "failureRate": 0.6,
  "errorMessage": "ChaosMesh-Lite: simulated upstream failure"
},
}
```

Rolling Circular Buffer

Fixed 50-item sample window for memory safety.

Dynamic Percentile Sort

In-place numeric sort computing true 95th percentile.

Terminal Logging Hooks

Zero-dependency stdout verification stream.

Continuous Streaming Lifecycle

1. Intercept & Stamp

High-resolution timer logs start and end elapsed duration.

2. Aggregate State

Update failure counter and compute running average.

3. Push Telemetry

Broadcast snapshot to browser client via WebSocket frame.



Zero-Polling Design: Event-driven architecture guarantees immediate visual refresh on client graphs.

Simulated Network Scenarios

Chaos Mode	Mechanism	Observed Impact	Client Outcome
Baseline Pass	Direct Proxy Pass	Latency < 15ms	200 OK Standard Payload
Delayed Jitter	setTimeout Pipeline	p95 Spike > 2000ms	Client Timeout Test
Fault Inject	HTTP 500 Intercept	68% Error Ratio	Retry Logic Activation
Socket Drop	req.socket.destroy()	Abrupt TCP Reset	ECONNRESET Handling

Modern Full-Stack Architecture



Node.js Core

Proxy engine with http-proxy.



React 18

Component state control plane.



Recharts

Live SVG latency curves.



Vite Bundler

Sub-second dev server reloads.

Demonstrated Resilience Testing

100%

Configurable Failure Reproducibility

Successfully tested client retry policies, circuit breakers, and connection pool resets under real network conditions.



ChaosMesh-Lite

Network Resilience & Fault Injection Control Plane



github.com/abhineetr10/chaosmesh-lite